

ILP-Based Scheduling for High-Level Synthesis

Milestone 4: Scientific Communication - Paper and Talk

Name	MD Rafiul Islam
Student ID	1232714
Course area	SS2026 ELE HW/SW Codesign Seminar
Topic	ILP-Based Scheduling for High-Level Synthesis
Document type	Final communication guide and submission checklist

Purpose: This document is a structured milestone artifact for understanding, analysis, evaluation, and scientific communication. It is written as technical notes, not as copied final prose.

1. Milestone 4 goal

Milestone 4 synthesizes the earlier milestones into scientific communication: the final paper and the talk. The paper should document the technical contribution, context, evaluation, and application example. The talk should explain the central idea clearly and defend the tradeoffs.

For the submitted GitHub milestone folder, this PDF is a communication guide/checklist, not a replacement for the final IEEE paper PDF itself.

2. Final paper message

Main thesis: ILP-based scheduling converts an HLS scheduling decision into a mathematical optimization problem. Binary variables choose operation start cycles; constraints enforce uniqueness, dependencies, and resource limits; the objective minimizes latency. The chosen schedule directly affects RTL behavior.

Paper section	Communication target
Introduction	Why HLS scheduling matters and why resource-latency tradeoff matters.
HLS overview	Explain behavioral input, operation graph, allocation, scheduling, binding, RTL output.
Scheduling problem	Show $y = a*b + c*d$ and dependencies $O1 \rightarrow O3$, $O2 \rightarrow O3$.
Mathematical basics	Define graph, resources, latency, and constrained optimization idea.
Formal ILP model	Present $x_{i,t}$, uniqueness, dependency, resource, and objective constraints.
Algorithm steps	Explain C++ exhaustive ILP-style search as a transparent demonstrator.
Results	Compare one multiplier and two multiplier schedules.
RTL realization	Explain VHDL FSM and Questa validation of $y = 26$.
Advantages and limitations	Balance optimality/explainability against scalability/model complexity.

3. 10-minute talk structure

Slide	Title	Core message
1	Motivation	Scheduling decides both time and hardware cost.
2	HLS flow	Behavioral algorithm becomes RTL through allocation, scheduling, binding.
3	Example graph	$O1$ and $O2$ are independent; $O3$ waits for both.

4	ILP idea	$x_{i,t}$ is the yes/no decision for operation i in cycle t .
5	Constraints	Uniqueness, dependencies, resources, and latency objective.
6	Algorithm	Generate candidates, check constraints, choose minimum latency.
7	Case 1 result	One multiplier serializes the multiplications: 3 cycles.
8	Case 2 result	Two multipliers execute in parallel: 2 cycles.
9	RTL validation	VHDL FSMs implement both schedules and Questa shows correct $y = 26$.
10	Conclusion	ILP is exact and explainable, but scalability must be considered.

4. Speaker script backbone

- Start with the design question: Should we buy time with extra hardware, or save hardware and accept more cycles?
- Explain that HLS scheduling is the point where this question becomes a formal decision.
- Use the example $y = a*b + c*d$ because it is small but shows independence, dependency, and resource conflict.
- Explain $x_{i,t}$ slowly: it is 1 only when operation i is placed in cycle t .
- Show why constraints are not decoration; they are the rules that prevent illegal hardware behavior.
- Compare the two results and say clearly: the same expression produces different schedules because allocation changes.
- Finish with balanced evaluation: good for exact exploration, not automatically scalable for large designs.

5. Likely discussion questions and safe answers

Question	Answer direction
Why use ILP if heuristics are faster?	Because ILP gives optimality for the formulated objective and is good for exact exploration or small critical kernels.
Is your C++ program a full ILP solver?	No. It is an ILP-style exhaustive demonstrator for a small graph. A full tool would call a professional solver.
What changes when two multipliers are available?	O1 and O2 are independent, so they can run in parallel; latency drops from 3 cycles to 2 cycles.
Does VHDL solve the scheduling problem?	No. VHDL realizes the selected schedules as RTL states and datapath operations.
What is the main limitation?	Scalability and model complexity for large graphs, loops, memory, branching, and multi-cycle operations.

6. Final submission checklist

- Paper uses IEEE template and is around the required seminar length.
- Every technical claim has a source or is clearly explained from the implementation result.
- Figures are referenced in the text and have clear captions.
- C++ implementation and VHDL/Questa result are consistent with the paper schedule table.
- AI usage protocol JSON and BibTeX entry are included for transparency.
- No AI-generated paragraphs are copied blindly; wording is reviewed and owned by the student.
- GitHub folders are named consistently: Milestone_1, Milestone_2, Milestone_3, Milestone_4.

Reference	How it supports this topic
Teich and Haubelt, 2007	German HLS / HW/SW synthesis background: allocation, scheduling, binding, optimization.
De Micheli, 1994	Main textbook-level foundation for synthesis and optimization of digital circuits.

McFarland, Parker, Camposano, 1990	Classic overview of high-level synthesis and behavioral synthesis flow.
Paulin and Knight, 1989	Neighboring heuristic approach: force-directed scheduling for behavioral synthesis.
Cong et al., 2011	Modern FPGA-oriented HLS context and deployment relevance.
Rettberg, 2026 seminar slides	Course requirements: scientific paper, C/C++ feature, application example, RTL component realization.